

CSCI-338-Computer Science Theory
Fall 2025
HW 05

Instructor: Adiesha Liyanage
Student Name:

This assignment is **due on November 18, 2025**. You will need to use Latex to generate a single pdf file and upload it under Assignment 05 on Canvas. There will be a penalty of not using Latex (to finish the assignment). This is not a group-assignment, so you must finish the assignment by yourself. **Figures can be hand drawn and inserted to the pdf.**

1 Problem 01

Provide a implementation-level description of TM that decides the following language: $L = \{\text{Same number of 0's and 1's}\}$.

When I say implementation level description, what I mean is for you to give an algorithm on how the Turing machine moves its head and way it is stored data when it decides the given language. Refer the implementation-level description related to $A = \{0^{2^n} | n \geq 0\}$. (You do not need to draw the Turing machine)

1. We give a one-tape deterministic Turing machine (TM) description using marking on the tape to pair up symbols. tape alphabet $\Gamma = \{0, 1, \#, X, Y, \sqcup\}$ where $\sqcup$ is blank, $\#$ is a left-end marker, X marks a crossed-out 0, and Y marks a crossed-out 1.

1. (Initialize) Move the head to the leftmost tape cell that contains input (optionally write a left sentinel ' $\#$ ' and move right).
2. Repeat:
 - (a) Scan right to find the first unmarked symbol in $\{0, 1\}$. If none found (all symbols are X or Y or blank), accept (every symbol was paired).
 - (b) If the symbol found is a 0, replace it with X (mark it). Then scan right to find an unmarked 1; if you find an unmarked 1, replace it with Y and then return the head to the leftmost position and go to step 2. If you scan the whole tape and do not find an unmarked 1, reject.

(c) If the symbol found is a 1, replace it with Y (mark it). Then scan right to find an unmarked 0; if you find one, replace it with X and return the head to the leftmost position and go to step 2. If you scan the whole tape and do not find an unmarked 0, reject.

- The machine uses only local marks (X, Y) and simple left-right scans; it requires no extra work tape beyond marking.
- Time complexity (number of head moves) is $O(n^2)$ in the worst case on an input of length n (each pairing may scan the whole tape).

2 Problem 02

Let $\Sigma = \{a, b\}$. We define the language $EVENT_{TM}$ as follows:

$$EVENT_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ contains only strings of even length.} \}$$

Prove that $EVENT_{TM}$ is undecidable.

2. Recall $A_{TM} = \{ \langle M, w \rangle \mid M \text{ accepts } w \}$ is undecidable. We reduce A_{TM} to $EVENT_{TM}$. Given input $\langle M, w \rangle$, construct a Turing machine M' that operates as follows on an input string x :

1. If $|x|$ is even, accept.
 2. If $|x|$ is odd, simulate M on w . If M accepts w , **accept**; otherwise do not accept.
- For all inputs of even length, M' always accepts, so the set of even-length strings is always a subset of $L(M')$.
 - For odd-length inputs, M' accepts them iff M accepts w .

Thus:

- If M accepts w , then M' accepts *every* even-length string and also every odd-length string, so $L(M')$ contains strings of both parities (in particular, odd length strings), so $L(M')$ is *not* a subset of the even-length strings. Therefore $\langle M' \rangle \notin EVENT_{TM}$.
- If M does not accept w (either rejects or loops), then M' accepts exactly the even-length strings (odd-length strings are not accepted), so $L(M')$ contains only even-length strings; hence $\langle M' \rangle \in EVENT_{TM}$.

Assume for contradiction that EVENT_{TM} were decidable. Then on input $\langle M, w \rangle$ we could compute $\langle M' \rangle$ and run the decider for EVENT_{TM} on it. The decider's answer would tell us whether M does *not* accept w ; by negating that answer we could decide whether M accepts w . That would decide A_{TM} , contradicting the known undecidability of A_{TM} . Therefore EVENT_{TM} is undecidable.

3 Problem 03

Show that EQ_{CFG} is undecidable.

3. For any CFG G over alphabet Σ , we can construct a CFG H that generates Σ^* (trivial to construct). Then $L(G) = \Sigma^*$ if and only if $L(G) = L(H)$.

So a decider for EQ_{CFG} would decide universality for CFGs, which is impossible.

Therefore EQ_{CFG} is undecidable.

4 Problem 04

Show that EQ_{CFG} is co-Turing recognizable.

4. If $L(G) \neq L(H)$ then there exists some string $w \in \Sigma^*$ such that $w \in L(G) \oplus L(H)$ (symmetric difference): that is, $w \in L(G)$ and $w \notin L(H)$ or vice versa. Membership of a string in a CFG is decidable (for example by the CYK algorithm after conversion to CNF). So we can enumerate all strings $w \in \Sigma^*$ (lexicographic or length-then-lexicographic order). For each enumerated w , decide whether $w \in L(G)$ and whether $w \in L(H)$. If for some w we find that membership differs (one accepts, the other does not), then output **accept** (we have witnessed $L(G) \neq L(H)$). If $L(G) = L(H)$, the enumeration will never find a witness and the recognizer will run forever.

This enumerator/algorithm recognizes precisely those pairs $\langle G, H \rangle$ with $L(G) \neq L(H)$. Therefore $\overline{\text{EQ}_{CFG}}$ is Turing-recognizable, and EQ_{CFG} is co-Turing-recognizable.

5 Problem 05

Find a match in the following instance of the Post Correspondence Problem.

$$\left[\frac{ab}{abab} \right], \left[\frac{b}{a} \right], \left[\frac{aba}{b} \right], \left[\frac{aa}{a} \right]$$

5. We search for a finite nonempty sequence of indices $i_1, i_2, \dots, i_k$ such that

$$\text{top}(i_1)\text{top}(i_2) \cdots \text{top}(i_k) = \text{bottom}(i_1)\text{bottom}(i_2) \cdots \text{bottom}(i_k).$$

One match is the sequence of indices (4,4,2,1)

Verify:

$$\begin{aligned} \text{top}(4)\text{top}(4)\text{top}(2)\text{top}(1) &= \text{a a a a b a b} = \text{a a a a b a b}, \\ \text{bottom}(4)\text{bottom}(4)\text{bottom}(2)\text{bottom}(1) &= \text{a a a a b a b} = \text{a a a a b a b}. \end{aligned}$$

6 Problem 06

6. Show that $ALL_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^* \text{ is decidable}\}$.

Given a DFA $A = (Q, \Sigma, \delta, q_0, F)$, define the complement DFA $A_{\text{comp}} = (Q, \Sigma, \delta, q_0, Q \setminus F)$ (same transition function and start state, accept states flipped). Observe $L(A) = \Sigma^*$ if and only if $L(A_{\text{comp}}) = \emptyset$.

So decision procedure:

1. Construct A_{comp} by flipping accepting states.
2. Run a BFS/DFS from q_0 . If an accepting state of A_{comp} is reachable, **reject** (so $L(A) \neq \Sigma^*$). If no such state is reachable, **accept** (so $L(A) = \Sigma^*$).

This procedure always halts on all strings.

7 Problem 07

7. Show that $A_{\epsilon CFG} = \{\langle G \rangle \mid G \text{ is CFG that generates } \epsilon\}$ is decidable.

Given CFG $G = (V, \Sigma, R, S)$, we compute the set Nullable $\subseteq V$ of variables that derive ϵ by fixed-point iteration:

1. Initialize Nullable₀ = $\{A \in V \mid A \rightarrow \epsilon \text{ is a production}\}$.
2. For $i \geq 0$ compute

$$\text{Nullable}_{i+1} = \text{Nullable}_i \cup \{A \in V \mid \exists A \rightarrow X_1 X_2 \cdots X_k \in R \text{ with each } X_j \in \text{Nullable}_i\}.$$

3. Continue until a fixed point Nullable_t is reached (no change).
4. Answer: $\langle G \rangle \in A_{CFG}$ iff $S \in \text{Nullable}_t$.

The procedure halts because V is finite and each step strictly increases the set or stops. Thus A_{CFG} is decidable.

8 Problem 08

8. Let $\mathcal{B}$ be the set of all infinite sequences over $\{0, 1\}$. Show that $\mathcal{B}$ is uncountable using proof by diagonalization.

Assume, toward contradiction, that B is countable. Then we can list all infinite binary sequences as a sequence:

$s_1, s_2, s_3, \text{each } s_i = (s_{i,1}, s_{i,2}, s_{i,3}, \dots)$ with $s_{i,j} \in \{0, 1\}$.

Define a new sequence $t = (t_1, t_2, t_3, \dots)$ by setting $t_j = 1 - s_{j,j}$,

that is, flip the j -th bit of the j -th listed sequence. By construction t differs from s_j in the j -th position, so $t \neq s_j$ for every j . Hence t is not in the enumerated list, contradicting the assumption that the list was exhaustive. Therefore B is uncountable.